Bio Bloom Performance Test Results (k6)

Checkout load test and subscription billing benchmark, run on 25 October 2026

Item	Detail
Project	Bio Bloom: Artisanal Food E-Commerce Platform
Course	SENG 34213
Department	Department of Software Engineering, Faculty of Science, University of Kelaniya
Document	Performance Test Results Report (k6)
Version	1.0
Date	25 October 2026
Status	Final for submission
Team	Ovindu Pathirana (SE/2021/059), Prageeth Ekanayake (SE/2021/049): backend, database, cron, security, DevOps. Chamindu Dilshan (SE/2021/058), Sanduni Ayeshika (SE/2021/039): frontend. I.G.R. Ravindu Laksara (SE/2021/028): mobile
Reference code	Monorepo `biobloom`: API `apps/api`, database `packages/prisma`, load tests `infra/load-test` (`k6/checkout.js`, `k6/subscription-cron.js`, `k6/lib.js`, `fixtures/due-subscriptions.sql`)
Related	Backlog ticket DDP-#38, SRS v2.0 NFR010 and NFR011 (`documents/01-SRS-Updated.md`), SDS v01 §5.3, §5.4 and §8.1, `infra/load-test/README.md`

Version history

Version	Date	Author	Summary
1.0	25 October 2026	Project team	First recorded k6 runs: checkout under load and billing job

Table of contents

1. Executive summary
2. Test environment
3. Method
4. Benchmark 1: checkout under load
5. Benchmark 2: daily subscription billing
6. Resource utilisation
7. Bottlenecks and optimisations
8. Limitations
9. Recommendations
10. Appendix A: commands used
11. Appendix B: raw data files

1. Executive summary

We ran the two k6 load tests from DDP-#38 against a local build of the API on 25 October 2026. Both benchmarks meet their targets with a large margin.

- **Checkout:** at up to 50 concurrent shoppers, the 95th-percentile (p95) checkout time was **48.19 ms** against a budget of 600 ms. **0.00%** of 28,349 requests failed. The API placed **9,280 orders** in 3 minutes.
- **Billing:** one billing run renewed **200 of 200** due subscriptions in **1.035 s** (server-side time) against a budget of 60 s, with 0 errors.
- **First checkout run failed.** The first run failed its `http_req_failed` threshold (98.88% failed requests). The cause was a defect in the test script, not in the API: k6 clears each virtual user's cookies after every iteration, but the script signs in only once per virtual user. The second run added the k6 option `noCookiesReset: true` and passed. The script in the repository still needs this fix (see §9).
- **Bottleneck:** the API process is the busiest component (CPU average 64.9%, peak 147.7% of one core). PostgreSQL (9.4% average) and Redis (4.3% average) were lightly loaded. No component was saturated at 50 shoppers.

Important caveat. The API, PostgreSQL, Redis and k6 all ran on one laptop. These numbers show how the system behaves relative to its budget. They are **not** a measure of production capacity. The staging environment on the Render free tier was **not** load-tested.

1.1 Results table

#	Benchmark	Target (source)	Measured	Result
1	Checkout p95 latency	p95 < 600 ms at 50 shoppers (DDP-#38 AC1, NFR010)	**48.19 ms** (avg 28.68 ms, max 327.64 ms)	**PASS**
1	Failed requests	< 1% (DDP-#38 AC1, NFR010)	**0.00%** (0 of 28,349)	**PASS**
1	Every checkout under 600 ms	Check in `checkout.js`	9,327 of 9,327 checkouts	**PASS**
1	First run (before script fix)	Failed requests < 1%	98.88% failed; only 50 checkouts succeeded	FAIL (test-script defect)
2	Billing run time	< 60 s for 200 due subscriptions (DDP-#38 AC3, NFR011)	**1.035 s** (k6 `billing_run_ms` max 1.03 s)	**PASS**
2	Every due subscription processed	200 found, 200 renewed, 0 errors (DDP-#38 AC3)	due 200, renewed 200, errored 0, chargeFailed 0	**PASS**
2	k6 checks	`checks rate==1`	100% (3 of 3)	**PASS**
2	No negative stock after the run	DDP-#38 AC3	0 batches with negative stock, queried after all runs (the fixture was reloaded between the billing and checkout runs). A database CHECK also forbids it (see §5.3)	**PASS** (measured after the checkout run)

1.2 Where the targets come from

SDS v01 states no latency figure. The budget comes from DDP-#38 (`docs/development-issues.md`) and `infra/load-test/README.md`, and it is recorded in the SRS v2.0 as NFR010 (checkout) and NFR011 (billing job). The SRS listed both as "Not measured" before these runs. This report supplies the first measurement.

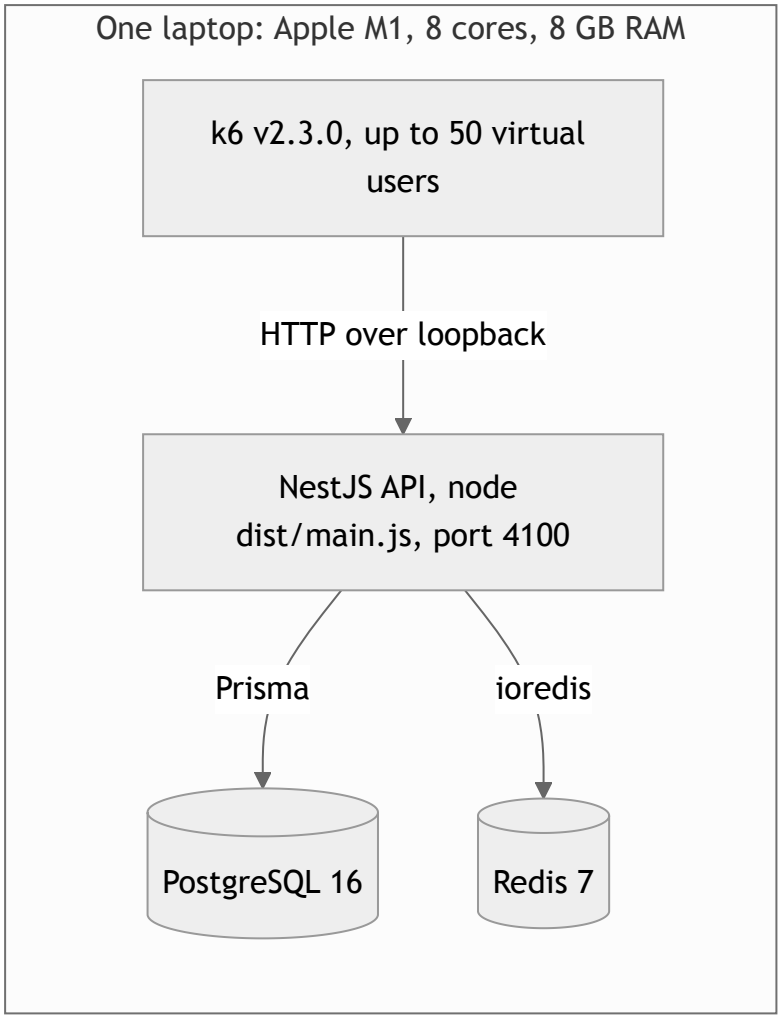
---

2. Test environment

2.1 Hardware and software

Component	Version or setting
Machine	One laptop, Apple M1 (8 cores), 8 GB RAM
Operating system	macOS 27
Runtime	Node.js 26.5
API build	`nest build`, then `node dist/main.js` (compiled build, not watch mode), port 4100
API settings	`NODE_ENV=development` (so the billing trigger is available), `PAYMENTS_MODE=mock`
Rate limits	Raised for the run: `THROTTLE_AUTH_LIMIT=1000`, `THROTTLE_DEFAULT_LIMIT=100000` (as `infra/load-test/README.md` says)
Database	PostgreSQL 16, local instance
Cache	Redis 7, local instance
Load generator	k6 v2.3.0, on the same machine

2.2 Topology



All traffic used the loopback interface, so there was no real network delay.

2.3 Test data

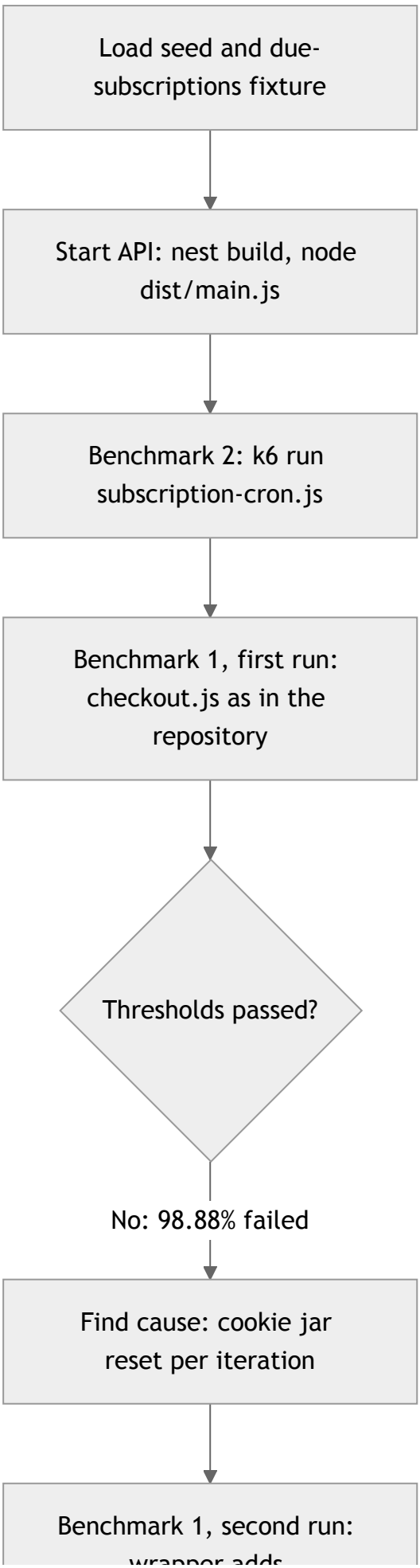
Data	Source	Content
Base catalogue, admin account, shipping rates	`packages/prisma/prisma/seed.ts`	Monorepo seed (products, variants, batches, 5 shipping brackets)
Load-test fixture	`infra/load-test/fixtures/due-subscriptions.sql`	200 customers `loadtest-001@example.com` to `loadtest-200@example.com`, one address each, one ACTIVE subscription each due today, and one 500-unit `LOADTEST` batch per active variant

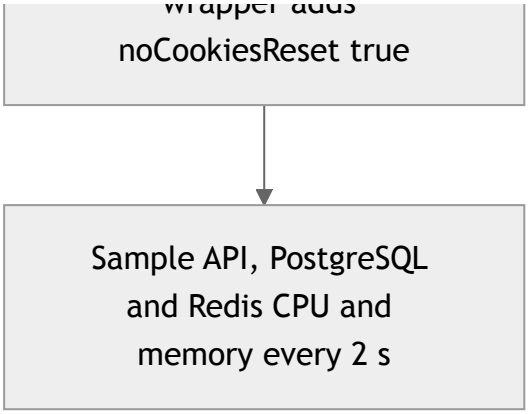
The fixture gives each virtual user its own customer and cart, so carts never collide. The FIFO row locks on shared product batches are what the checkout test really measures.

---

3. Method

3.1 Order of the runs


Load seed and due-subscriptions fixture
Start API: nest build, node dist/main.js
Benchmark 2: k6 run subscription-cron.js
Benchmark 1, first run: checkout.js as in the repository
Thresholds passed?
No: 98.88% failed
Find cause: cookie jar reset per iteration
Benchmark 1, second run:

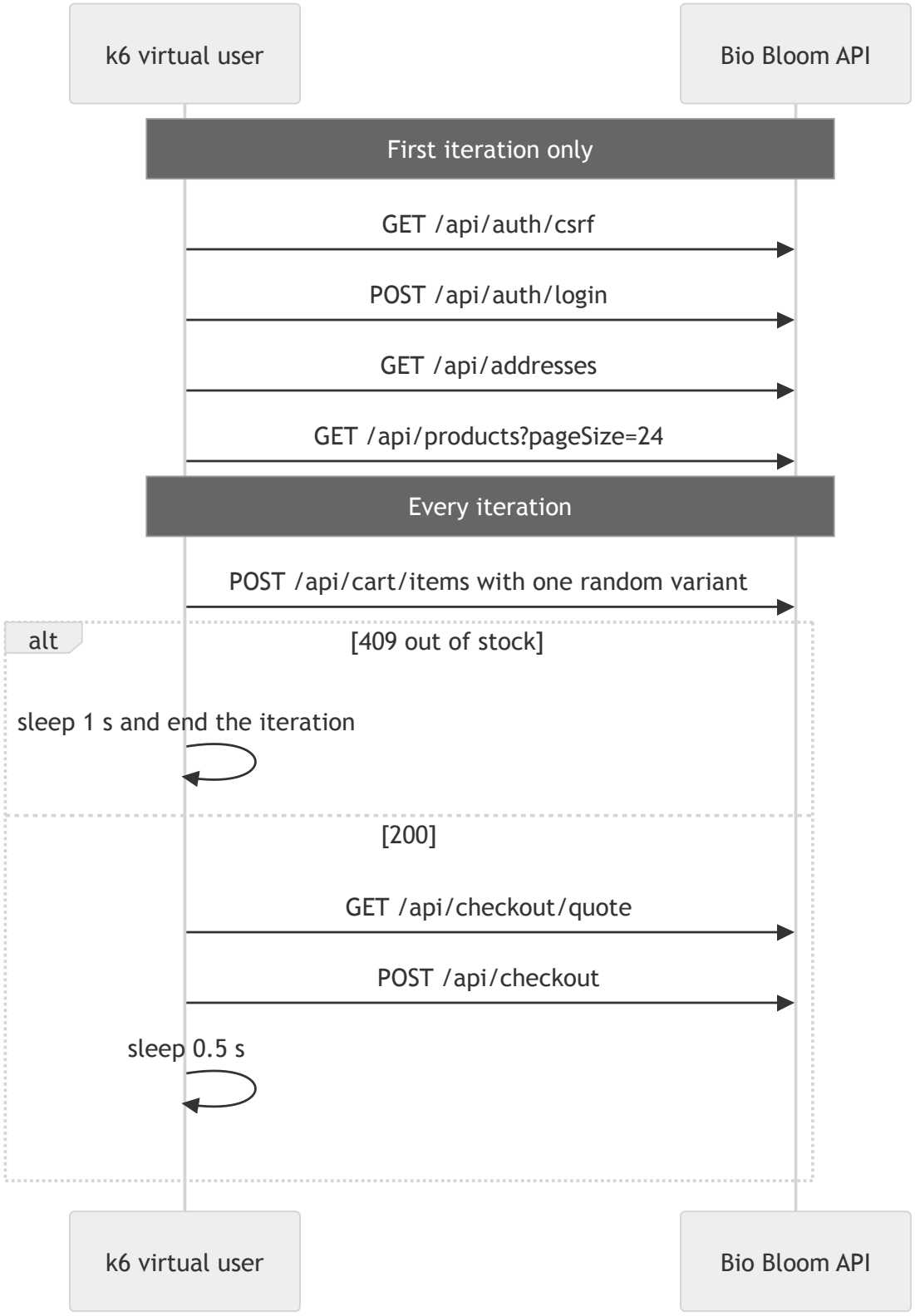


The billing test ran first, while the 200 subscriptions were still due, as the README asks. The checkout test then reused the same 200 customers.

3.2 Benchmark 1 workload (`infra/load-test/k6/checkout.js`)

Setting	Value
Executor	`ramping-vus`, scenario `shoppers`
Stages	0 to 25 virtual users (VUs) in 30 s, then to 50 over 2 min, then to 0 in 30 s (3 min total)
Graceful ramp-down	15 s
Thresholds	`http_req_duration{stage:checkout}: p(95)<600`, `http_req_failed: rate<0.01`
Expected statuses	200, 201 and 409 on add-to-cart and checkout (409 means "out of stock" and is a correct answer)

Each VU does the following:



3.3 Benchmark 2 workload (`infra/load-test/k6/subscription-cron.js`)

One VU signs in as the seeded admin and calls `POST /api/admin/subscriptions/billing/run` once. This endpoint runs the same `BillingCronService.runBilling` code as the 06:00 UTC cron and returns the counts and `durationMs` (`apps/api/src/subscriptions/subscriptions-admin.controller.ts`, lines

52 to 60). It returns 404 when `NODE_ENV=production`. Thresholds: `billing_run_ms max<60000` and `checks rate==1`.

3.4 Resource sampling

During the second checkout run, the CPU % and resident memory (RSS) of the API process, the PostgreSQL processes and the Redis process were sampled every 2 seconds. This gave 87 samples over the 3-minute run. CPU % is per core: 100% is one full core and 800% is all 8 cores.

3.5 Metrics used

Metric	Meaning
`http_req_duration{stage:checkout}`	Time for `POST /api/checkout`, which is the budgeted request
`http_req_duration`	Time for every HTTP request in the test
`http_req_failed`	Share of responses with a status the script does not expect
`checkout_latency_ms`	Custom k6 trend, same values as the checkout request duration
`billing_run_ms`	Server-side billing run time, from `durationMs` in the response

---

4. Benchmark 1: checkout under load

4.1 First run: failed because of a test-script defect

The first run used `infra/load-test/k6/checkout.js` exactly as it is in the repository.

Measure	Value
`http_req_failed`	**98.88%**, threshold FAILED
Checkouts that succeeded	50 (one per VU)
Error statuses	401 and 403 after the first iteration

Cause. `signIn()` in `infra/load-test/k6/lib.js` returns only the request headers (`Content-Type` and `X-CSRF-Token`). The session itself is in cookies that k6 stores in the VU's cookie jar. By default k6 **resets the cookie jar at the start of every iteration**. The script signs in only on the first iteration (`if (!session) setUpVu();`). So from the second iteration on, every request went out with no session cookie and no CSRF cookie. The API correctly answered 401 (not signed in) or 403 (CSRF check failed).

Conclusion. This is a defect in the test script, not in the API. The API did what it should: it refused requests that had no valid session.

Fix. Keep the cookie jar between iterations with the k6 option `noCookiesReset: true`. For the second run we used a small wrapper that imports the unchanged script and adds only this option:

```
import run, { options as base } from 'file:///path/to/biobloom/infra/load-
test/k6/checkout.js';
export const options = Object.assign({}, base, { noCookiesReset: true });
export default run;
```

The stages, thresholds, checks and request code are the same as in the repository script.

4.2 Second run: headline results

Measure	Value	Threshold	Result
Checkout p95 (`stage:checkout`)	**48.19 ms**	< 600 ms	**PASS**
Checkout average / median / max	28.68 ms / 28.81 ms / 327.64 ms		
All requests p50 / p95	17.44 ms / 51.12 ms		
`http_req_failed`	**0.00%** (0 of 28,349)	< 1%	**PASS**
HTTP requests	28,349 (157.3 req/s average)		
Iterations	9,495 (52.7 per second)		
Orders placed (201)	9,280		
VUs	`vus_max` 50		
k6 checks	99.42% (37,261 of 37,476)	not a threshold	
Data received / sent	37 MB / 18 MB		

The checkout p95 used **about 8%** of the 600 ms budget (48.19 / 600). The slowest single checkout (327.64 ms) was still under the budget, and the check "checkout under 600 ms" passed for all 9,327 checkout requests.

The checks rate is below 100% only because of expected 409 answers. The checks "added to cart" and "order placed" count only 200 and 201 as passes, while the script lists 409 as an expected status, so 409 does not count as a failed request.

4.3 Status codes per endpoint

From `k6-status-breakdown.tsv`:

Endpoint	Status	Count	Note
`POST /api/cart/items`	200	9,327	Item added
`POST /api/cart/items`	409	168	Variant out of stock (expected)
`GET /api/checkout/quote`	200	9,327	Quote returned
`POST /api/checkout`	201	9,280	Order placed

Endpoint	Status	Count	Note
`POST /api/checkout`	409	47	Stock ran out between cart add and checkout (expected)
`POST /api/auth/login`	200	50	One sign-in per VU
`GET /api/auth/csrf`	200	50	One per VU
`GET /api/addresses`	200	50	One per VU
`GET /api/products`	200	50	One per VU. The status column is empty for this row in `k6-status-breakdown.tsv`; the raw CSV shows all 50 as 200
Total		**28,349**	

Why 409s appeared. 9,280 orders of one unit each drew down the 500-unit `LOADTEST` batch (plus the seeded batches) for some variants until they were empty. After that, add-to-cart and checkout for those variants correctly returned 409 instead of selling stock that does not exist. The 47 checkout 409s show the FIFO decrement inside the transaction rejecting an order when stock ran out between the cart check and the checkout. This is the intended behaviour (`apps/api/src/inventory/inventory.service.ts`, lines 136 to 139).

4.4 Response-time distribution

The first two rows are from the k6 end-of-test summary (`k6-checkout.txt`, `k6-checkout-summary.json`). The per-endpoint rows were computed from the raw k6 CSV output (`k6-checkout-raw.csv`) of the same run. Their totals match the summary.

Request	Count	p50 (ms)	p90 (ms)	p95 (ms)	Max (ms)
All requests	28,349	17.44	42.87	51.12	331.52
`POST /api/checkout` (budgeted)	9,327	28.81	42.62	**48.19**	327.64
`POST /api/cart/items`	9,495	26.97	54.38	148.70	314.91
`GET /api/checkout/quote`	9,327	11.18	16.90	18.65	302.77
`POST /api/auth/login`	50	308.73	316.31	317.15	331.52
`GET /api/products?pageSize=24`	50	12.24	19.01	19.90	22.35
`GET /api/addresses`	50	3.53	6.57	7.06	10.36
`GET /api/auth/csrf`	50	1.97	3.13	3.60	4.28

The p99 for all requests was 220.01 ms (from the raw CSV).

What this shows:

- **Sign-in is the slowest request** (median 308.73 ms). It runs bcrypt with cost factor 12 (`apps/api/src/auth/auth.service.ts`, line 20). This cost is deliberate for password security (SDS §7.3). It happens once per VU, so it barely affects the totals.

- **Add-to-cart has the longest tail** among the repeated requests (p95 148.70 ms against a median of 26.97 ms). See §7.1.
- **The quote is fast** (p95 18.65 ms). It uses the cached shipping brackets (see §7.2).

Each iteration took 587.96 ms on average. 500 ms of that is the script's own `sleep(0.5)`, so the three requests of a normal iteration took about 88 ms together.

4.5 Response time over time

Per 10-second window, from `k6-timeseries.tsv`:

Window start (s)	req/s	p50 all (ms)	p95 all (ms)	Checkout p95 (ms)
0	24.8	6.9	176.2	17.9
10	63.3	14.4	154.6	27.9
20	109.4	23.2	109.0	31.8
30	126.5	29.5	43.2	37.3
40	139.4	30.8	45.9	38.5
50	152.9	33.2	49.4	47.4
60	165.5	25.9	44.0	39.8
70	164.0	31.9	53.5	41.0
80	184.7	28.8	54.1	46.9
90	197.9	15.3	42.6	39.5
100	228.0	11.1	30.1	40.7
110	219.1	16.6	41.1	42.5
120	217.4	35.7	55.7	54.7
130	256.2	13.7	40.7	49.8
140	253.1	18.1	51.9	53.0
150	205.7	11.0	49.4	51.2
160	104.6	6.4	15.4	17.9
170	22.4	5.1	12.1	12.2

Chart 1. Response time over time (ms). Line 1: p95 of all requests. Line 2: checkout p95. Line 3: p50 of all requests. The 600 ms budget is far above the top of this chart.

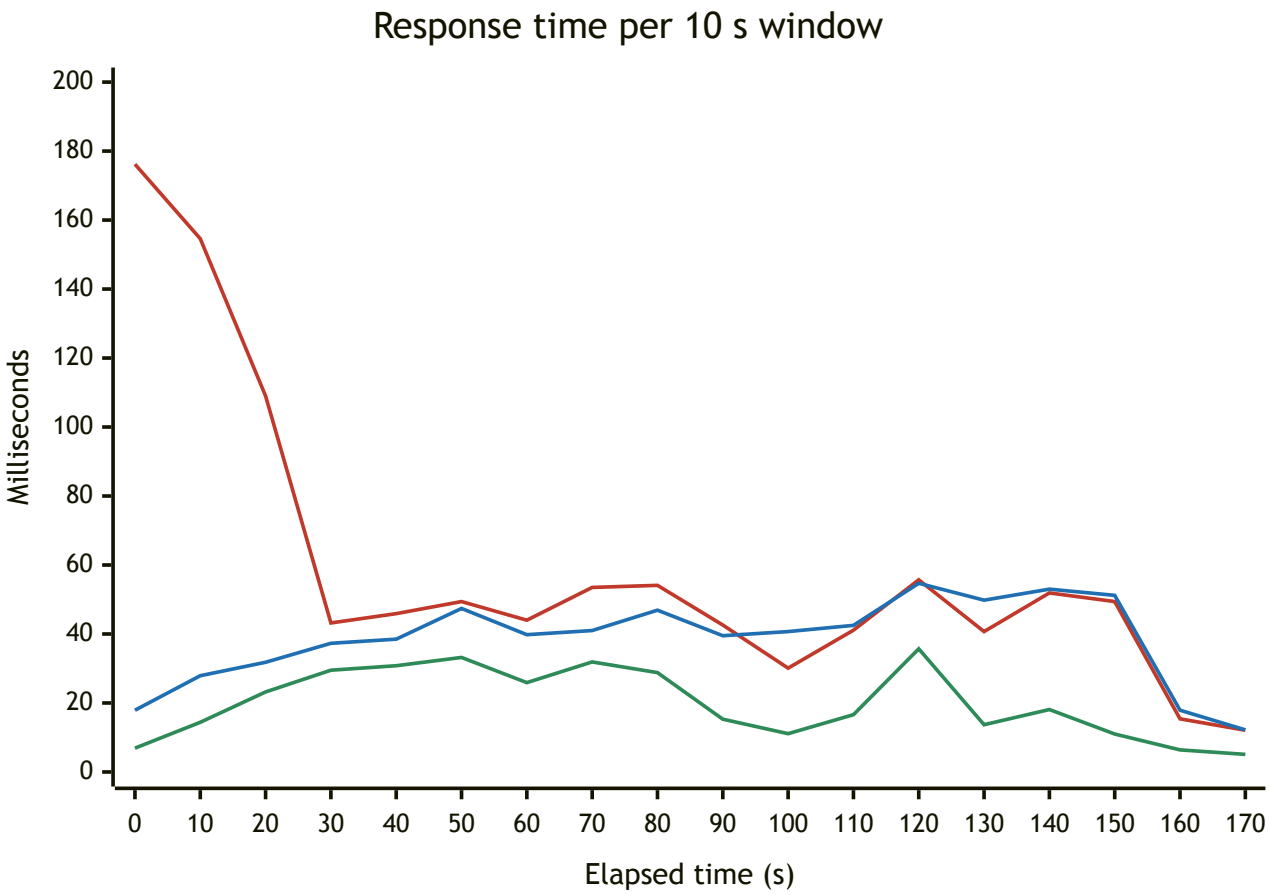
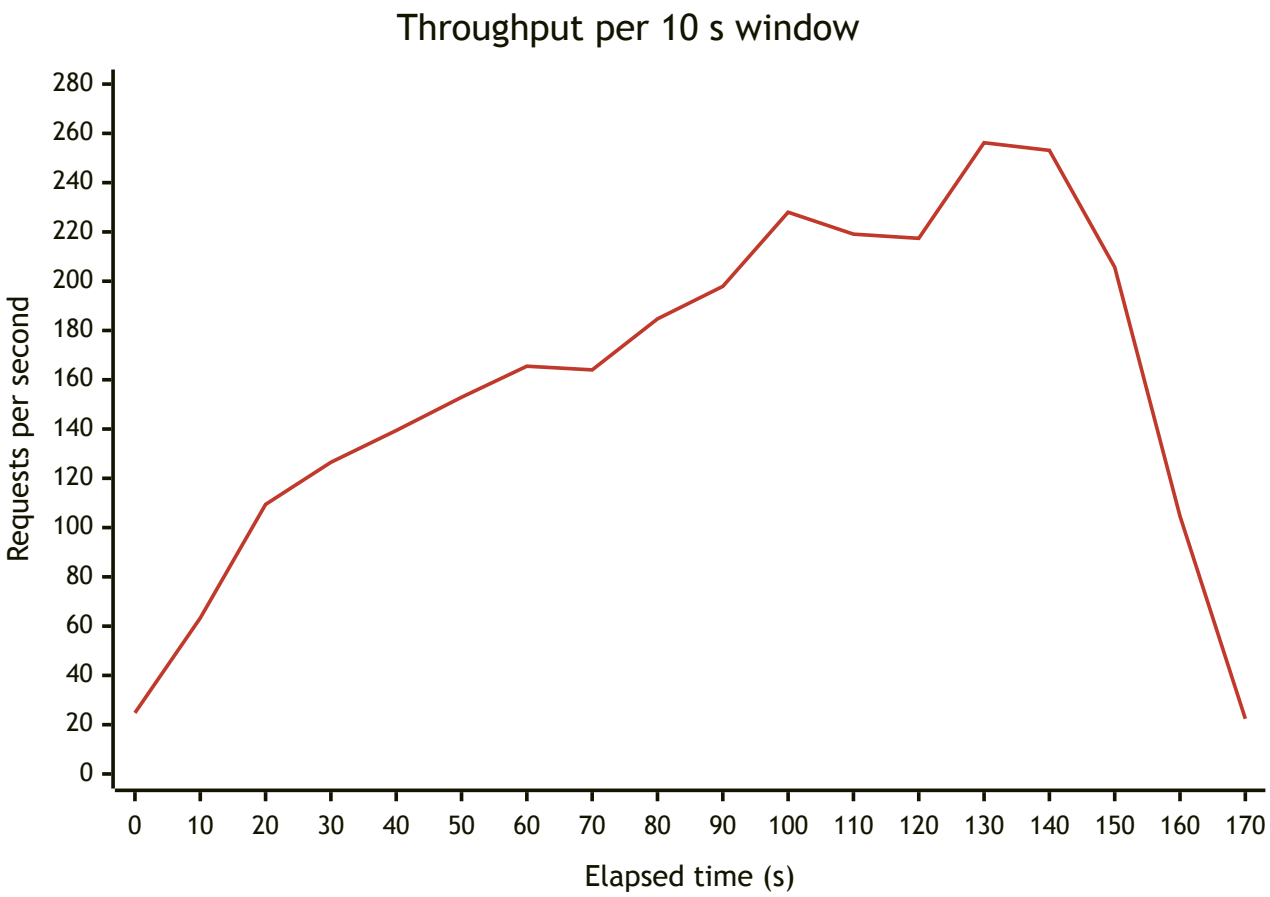


Chart 2. Throughput over time (requests per second).



Observations:

1. **Warm-up spike.** In the first 20 s, the p95 of all requests was 109 to 176 ms, while the checkout p95 stayed at 17.9 to 31.8 ms. The raw CSV shows 8 to 9 sign-ins (about 300 ms each) in each of the first three 10-s windows, against 0 to 3 later, and a high add-to-cart p95 (118 to 177 ms) in the same windows. Sign-ins land in the slowest 5% when there are only a few hundred requests in a window. From 30 s on, the p95 of all requests stayed between 30 and 56 ms.
2. **Throughput follows the VU count.** It rose from 24.8 req/s to a peak of **256.2 req/s** (window 130 s) as VUs grew to 50, and fell during the ramp-down. Throughput did not flatten before the ramp-down, so the API was not yet at its limit.
3. **Checkout time grows slowly with load.** The checkout p95 rose from 17.9 ms at the start to a highest window value of 54.7 ms (window 120 s). That is still under one tenth of the budget.
4. The p50 moved between 11 and 36 ms during the plateau. We did not analyse the cause.

---

5. Benchmark 2: daily subscription billing

5.1 Results

From `k6-billing.txt` and `k6-billing-summary.json`. The API returned:

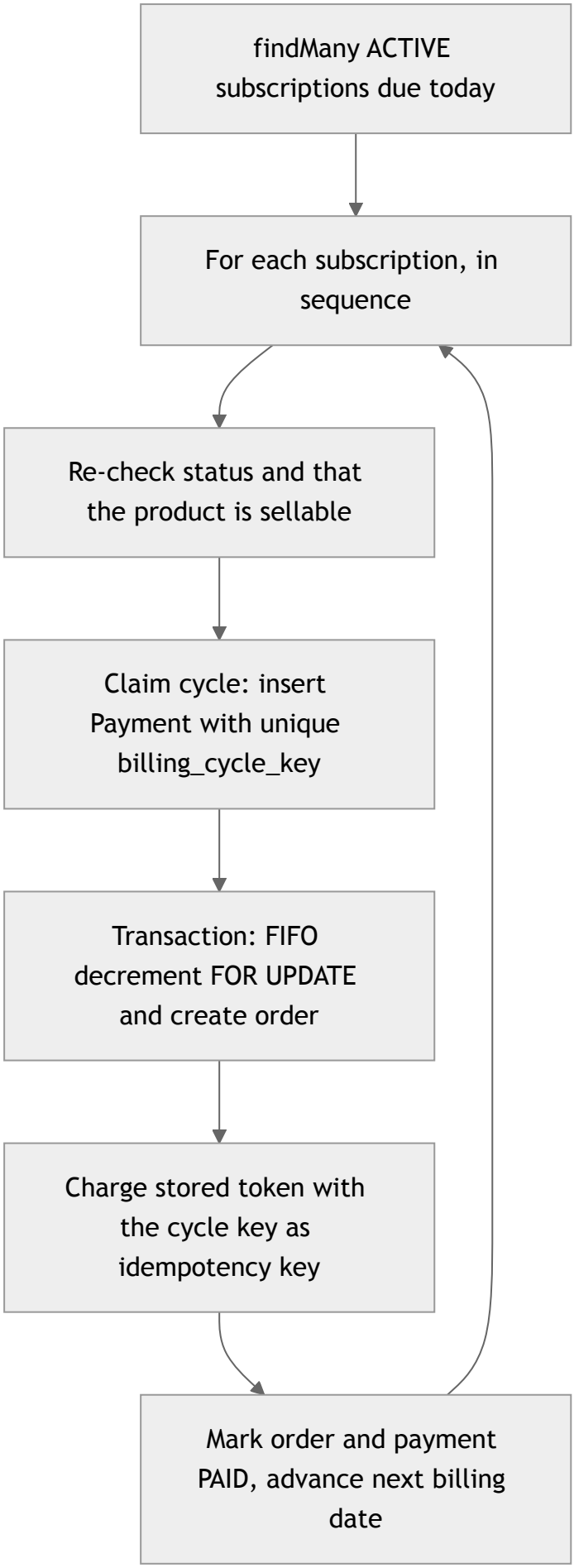
{"skipped":0,"errored":0,"advanced":200,"durationMs":1035,"due":200,"renewed":200,"outOfStock":0,"chargeFailed":0}

Measure	Value	Threshold or check	Result
Server-side run time (`durationMs`)	**1,035 ms (1.035 s)**	`billing_run_ms max<60000`	**PASS** (k6 shows max 1.03 s)
Due subscriptions found	200	`due >= 200`	**PASS**
Renewed	200	`renewed >= 200` and `errored == 0`	**PASS**
Errored / charge failed / out of stock / skipped	0 / 0 / 0 / 0		
Billing dates advanced	200		
k6 checks	100% (3 of 3)	`checks rate==1`	**PASS**
k6 iteration (sign-in plus run)	1.34 s		
HTTP requests	3 (CSRF, login, billing run), 0 failed		

The run used **1.7%** of the 60 s budget (1.035 / 60). That is about **5.2 ms per subscription**. The API log for the same run records `Billing tick: due=200 renewed=200 outOfStock=0 chargeFailed=0 skipped=0 errored=0`.

5.2 What one billing run does

`BillingCronService.runBilling` (`apps/api/src/subscriptions/billing-cron.service.ts`, lines 75 to 107) loads every ACTIVE subscription that is due, then processes them **one after another** in a `for` loop (line 92). For each one it claims the cycle with a unique `billing_cycle_key` payment row, runs the FIFO decrement and order insert in one transaction, and charges the stored token through the mock gateway.


findMany ACTIVE subscriptions due today
For each subscription, in sequence
Re-check status and that the product is sellable
Claim cycle: insert Payment with unique billing_cycle_key
Transaction: FIFO decrement FOR UPDATE and create order
Charge stored token with the cycle key as idempotency key
Mark order and payment PAID, advance next billing date

5.3 Acceptance criteria check

DDP-#38 AC3 item	Evidence	Status
Run completes in under 60 s	`durationMs` 1035	Met
Every subscription processed	due 200, renewed 200, errored 0	Met
Processed exactly once	One run only. A second charge for the same cycle is blocked by the unique `billingCycleKey` (`packages/prisma/prisma/schema.prisma`, line 423). A repeat run was not tested	Met for this run, repeat not tested
No stock quantity is negative afterwards	`SELECT count(*) FROM product_batch WHERE current_quantity < 0` returned 0 after all runs (the fixture was reloaded between the billing and checkout runs, so this covers the checkout run). The database also rejects a negative value: `CHECK ("current_quantity" >= 0 ...)` in `packages/prisma/prisma/migrations/20261022090000_spec_completion/migration.sql`, lines 59 to 60	Measured: 0 (after the checkout run)

---

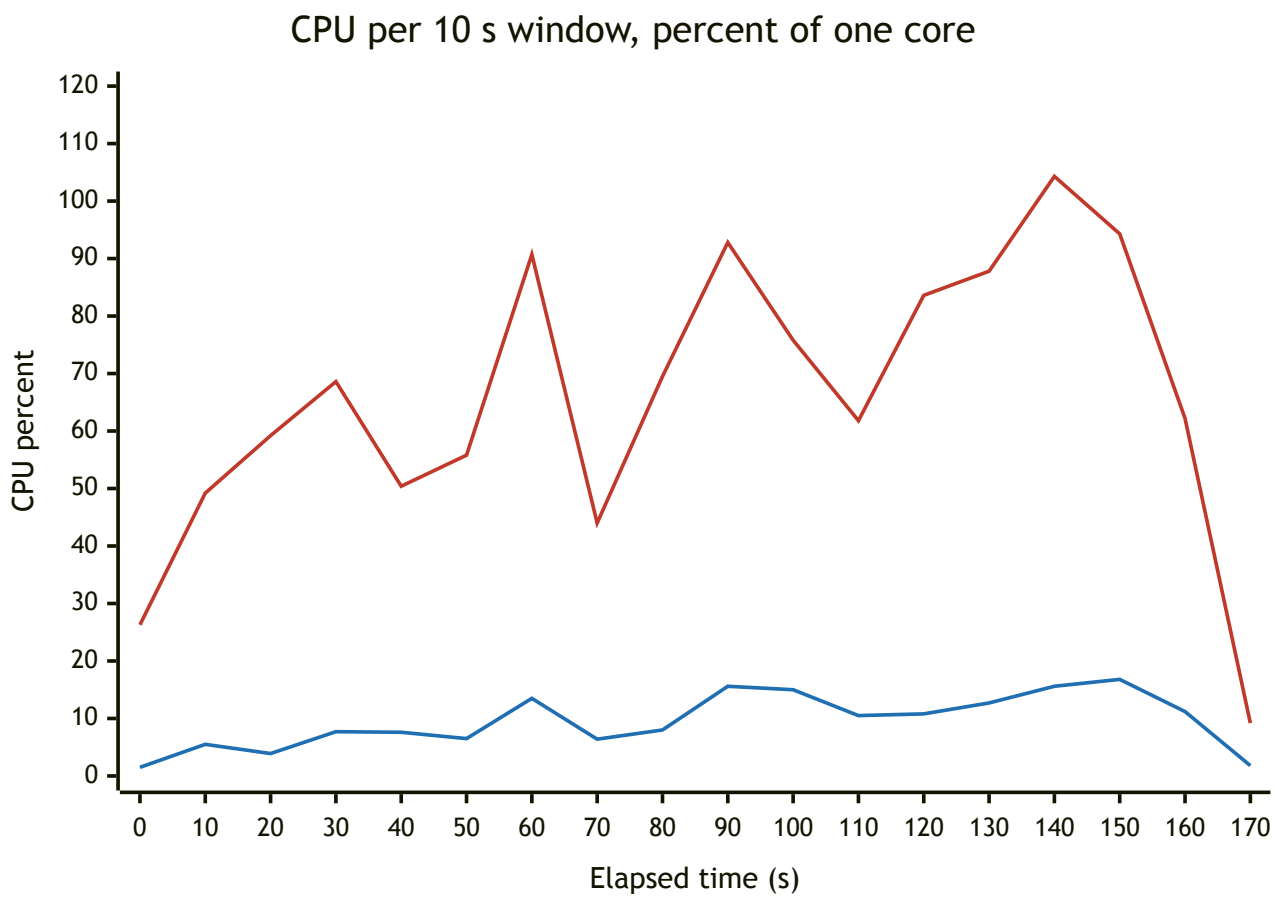
6. Resource utilisation

87 samples, one every 2 seconds, during the second checkout run (`resource-summary.txt`, raw values in `resource.csv`).

Component	CPU average	CPU max	Memory (RSS) average	Memory (RSS) max	Note
API (`node dist/main.js`)	**64.9%**	**147.7%**	230 MB	301 MB	Busiest component
PostgreSQL (all processes)	9.4%	25.1%	387 MB (sum)	482 MB (sum)	The sum counts shared memory more than once, so the real figure is lower
Redis	4.3%	11.8%	not summarised	3.5 MB	

CPU % is per core: 100% is one full core and 800% is all 8 cores. At its peak, the API used under one fifth of the machine's total CPU.

Chart 3. CPU over time (% of one core, average per 10 s window, computed from `resource.csv`). Line 1: API process. Line 2: PostgreSQL.



Observations:

- **API CPU follows the load.** The 10-s average rose from 26.3% at the start to 104.3% in the 140 s window, when 50 VUs were active and throughput was near its peak. The highest single sample was 147.7% (about 92 s into the run).
- **API CPU above 100%.** Node.js runs application JavaScript on one main thread. Values above 100% come from work on other threads of the same process, such as the libuv thread pool that runs bcrypt and the garbage collector. The main thread itself cannot use more than one core. This is why the API is the first component that will limit throughput.
- **PostgreSQL was lightly loaded.** Its 10-s average never went above 16.8%, even with about 50 orders per second, each with a `FOR UPDATE` row lock.
- **Redis was lightly loaded** (4.3% average). It serves the JWT revocation check, the account cache, the rate-limit counters and the shipping-bracket cache.
- **API memory was stable** within this 3-minute run: 178 MB in the first sample (182,640 KB), 301 MB at the highest and 220 MB (225,152 KB) in the last sample. A 3-minute run is too short to rule out a slow leak.

---

7. Bottlenecks and optimisations

7.1 Bottlenecks found from the data

#	Finding	Evidence	Impact at 50 shoppers
B1	**API CPU is the main constraint**	API 64.9% average, 147.7% max of one core. PostgreSQL 9.4%, Redis 4.3%. Throughput still rising when the ramp-down began	Not saturated. It is the first resource that will saturate as load grows
B2	**Warm-up p95 spike in the first 20 s**	p95 of all requests 176.2, 154.6 and 109.0 ms in the first three windows, then 30 to 56 ms. 8 to 9 sign-ins per window at the start (median 308.73 ms each, bcrypt cost 12). Add-to-cart p95 118 to 177 ms in the same windows	Short. Checkout p95 was not affected (17.9 to 31.8 ms)
B3	**Add-to-cart tail latency**	p95 148.70 ms against a median of 26.97 ms. Per the code, `CartService.addVariant` makes about six database calls in sequence: variant lookup, existing line, rest-of-cart demand, stock aggregate, upsert, then `getCart` with one stock query per line (`apps/api/src/cart/cart.service.ts` , lines 214 to 243 and 280 to 294)	Highest p95 of the repeated requests. The exact cause of the tail was **not profiled**
B4	**Row-lock cost of the FIFO decrement**	Checkout runs `SELECT ... FOR UPDATE` on the variant's batches inside a transaction (`apps/api/src/inventory/inventory.service.ts` , lines 101 to 115), so checkouts of the same variant wait for each other. Checkout p95 rose from 17.9 ms to at most 54.7 ms as VUs grew	Small: p95 48.19 ms. Lock waits were not measured on their own
B5	**Per-request log lines**	`CartService` , `InventoryService` and `OrdersService` each write one LOG line per cart add, decrement and checkout. The API log of the test session holds 9,377, 9,531 and 9,330 such lines	Cost not measured
B6	**Sequential billing loop**	`runBilling` processes subscriptions one at a time (`billing-cron.service.ts` , line 92). About 5.2 ms each locally	None at 200 subscriptions (1.035 s of 60 s). Time will grow in line with the number of due subscriptions

7.2 Optimisations that exist in the code

Only items that exist in the code are listed here.

#	Optimisation	Where	Effect seen in this test
O1	**Redis cache of shipping-rate brackets**, 5-minute TTL (SDS §5.3)	`apps/api/src/shipping/shipping.service.ts` , lines 5 to 6 (`shipping:rates` , 300 s) and lines 72 to 99. Admin edits clear it (`shipping-rates-admin.service.ts` , line 108)	The API log shows the brackets loaded from the database only once during the checkout run (07:15:32 UTC), while the run made about 18,654 shipping calculations (9,327 quotes and 9,327 checkouts). Quote p95 18.65 ms

#	Optimisation	Where	Effect seen in this test
O2	**Account cache for authenticated requests**, 60 s TTL	`apps/api/src/auth/strategies/jwt.strategy.ts`, line 18 and lines 78 to 89	Avoids a database lookup of the customer on every authenticated request. Not measured on its own
O3	**Short checkout transaction**	`apps/api/src/orders/orders.service.ts`, lines 174 to 193: address check, cart load, promo preview and shipping quote run **before** `$transaction`, so row locks cover only the decrement, order insert and cart clear	Keeps lock time short (B4)
O4	**Memoised stock lookups in one cart render**	`apps/api/src/cart/cart.service.ts`, lines 358 to 369 (`stockLookup`)	One stock query per variant per render, not per mention
O5	**Indexes on the hot paths**	`packages/prisma/prisma/schema.prisma`: `ProductBatch @@index([productVariantId, expiryDate])` line 230 (FIFO query), `@@index([status, expiryDate])` line 231, `Subscription @@index([status, nextBillingDate])` line 352 (billing query), `CartItem @@unique([customerId, productVariantId])` line 650 (cart upsert), `@@index([customerId])` line 652, `Order @@index([customerId, placedAt])` line 303	PostgreSQL CPU stayed at 9.4% average
O6	**Redis cron lock** (`SET ... NX EX`)	`apps/api/src/common/locks/cron-lock.service.ts`, lines 38 to 55; used by `billing-cron.service.ts`, lines 69 to 72	A correctness and scaling guard, not a speed-up: with several API instances only one runs the billing job. The load test calls `runBilling` directly, so the lock was not part of the measurement
O7	**Idempotent billing cycle**	Unique `billingCycleKey` on `Payment` (`schema.prisma`, line 423)	A restarted or concurrent run cannot charge the same cycle twice
O8	**Shared rate-limit counters in Redis**	`apps/api/src/app.module.ts`, lines 58 to 75	A protection, not a speed-up. Limits were raised for this test

7.3 Not implemented

These are common optimisations that the code does **not** have today:

- No HTTP response compression (no `compression` package in `apps/api/package.json`).
- No cache for product listings (`apps/api/src/products/products.service.ts` reads the database on every request).
- No tuned Prisma connection pool: no `connection_limit` is set in the repository, so the Prisma default applies.
- No parallel processing in the billing loop.

None of these was needed to meet the budget at 50 shoppers.

---

8. Limitations

#	Limitation	Effect on the results
L1	All components ran on one laptop (API, PostgreSQL, Redis and k6)	k6 competed with the API for CPU. Numbers show relative behaviour, **not production capacity**
L2	Loopback network	No network delay. Real users see higher times
L3	Staging (Render free tier and Vercel) was **not** load-tested	No measurement exists for the deployed system
L4	`PAYMENTS_MODE=mock`	No Stripe or PayHere network time in checkout or billing
L5	`NODE_ENV=development`, needed for the billing trigger	Not identical to a production configuration
L6	Rate limits raised (`THROTTLE_AUTH_LIMIT=1000`, `THROTTLE_DEFAULT_LIMIT=100000`)	The SDS §7.4 limits were not part of the test
L7	One run of each benchmark after the fix; 3-minute checkout run	No repeat runs, so no spread between runs. No soak test for leaks
L8	Peak of 50 VUs only	The breaking point was not found
L9	Small data set (seed plus 200 fixture customers), one item per order, no gift boxes or promo codes	Larger carts and tables may behave differently
L10	Resources sampled every 2 s; PostgreSQL RSS double-counts shared memory	Short spikes may be missed; PostgreSQL memory is overstated
L11	Billing: one run, 200 subscriptions, mock gateway, no declined charges	The failure paths (stock delay, declined charge) were not timed
L12	The repository script `checkout.js` still has the cookie defect; the passing run used a wrapper	The repository script fails as it is until the fix in R1 is merged
L13	No front-end performance test (for example page load) was run	Not measured

9. Recommendations

#	Recommendation	Why	Priority
R1	**Fix the cookie option in the repository.** Add `noCookiesReset: true` to `options` in `infra/load-test/k6/checkout.js` (snippet below), then re-run to confirm	The script as committed fails with 98.88% failed requests	High
R2	**Add k6 to CI.** Add a job to `.github/workflows/ci.yml` (no k6 step exists today) that starts PostgreSQL and Redis services, loads the seed and fixture, runs both scripts with a shorter profile, and fails the build when a threshold fails. Run the full profile nightly	Catches performance regressions before merge (DDP-#38 AC2)	High

#	Recommendation	Why	Priority
R3	**Run against staging on a paid instance.** The Render free tier sleeps and shares CPU, so a result there would mostly measure the platform. Use a paid instance for the test and run `checkout.js` with `-e API_BASE=https://biobloom-api.onrender.com`. Update the staging URL in `infra/load-test/README.md`, which still shows `api.staging.biobloom.lk`	First real measurement of the deployed system	High
R4	**Check AC3 after each billing run** with SQL: no negative `current_quantity`, and one `PAID` payment per `billing_cycle_key`	Turns "enforced by schema" into a measured result	Medium
R5	**Run k6 from a separate machine**	Removes CPU competition between k6 and the API	Medium
R6	**Add a stress test and a soak test.** Raise VUs until checkout p95 passes 600 ms to find the limit; run 30 to 60 minutes at 50 VUs to watch API memory and the connection pool	Finds the real capacity and slow leaks	Medium
R7	**Profile add-to-cart** before changing it, then reduce sequential queries if the profile confirms them	Longest tail of the repeated requests (B3)	Low
R8	**Measure logging cost** and move the per-request LOG lines to `debug` if it is significant	Unknown cost at higher load (B5)	Low
R9	**Time a larger billing run** (for example 2,000 subscriptions) and a run with declined charges	Confirms linear growth of the sequential loop (B6)	Low
R10	**Repeat each run three times** and report the median, and record the k6 exit code	Shows the spread between runs	Low

Snippet for R1, in `infra/load-test/k6/checkout.js`:

```
export const options = {
  noCookiesReset: true,
  scenarios: {
    shoppers: {
      executor: 'ramping-vus',
      startVUs: 0,
      stages: [
        { duration: '30s', target: 25 },
        { duration: '2m', target: 50 },
        { duration: '30s', target: 0 },
      ],
      gracefulRampDown: '15s',
    },
  },
  thresholds: {
    'http_req_duration{stage:checkout}': ['p(95)<600'],
    http_req_failed: ['rate<0.01'],
  },
};
```

Queries for R4:

```
SELECT count(*) AS negative_batches FROM product_batch WHERE current_quantity < 0;

SELECT billing_cycle_key, count(*) FROM payment
WHERE billing_cycle_key IS NOT NULL
GROUP BY billing_cycle_key HAVING count(*) > 1;
```

---

10. Appendix A: commands used

Run from the repository root unless stated. The values below are the local, test-only settings used for the run.

Build and start the API

```
cd apps/api
pnpm exec nest build
NODE_ENV=development PAYMENTS_MODE=mock THROTTLE_AUTH_LIMIT=1000
THROTTLE_DEFAULT_LIMIT=100000 PORT=4100
DATABASE_URL=postgresql://biobloom@127.0.0.1:5434/biobloom REDIS_URL=redis://localhost:6380
SUBSCRIPTION_TOKEN_ENC_KEY=loadtest-subscription-token-key-0001 node dist/main.js
```

Load the data

```
pnpm --filter @biobloom/prisma seed
psql -h 127.0.0.1 -p 5434 -U biobloom -d biobloom -v enc_key=loadtest-subscription-token-key-0001 < infra/load-test/fixtures/due-subscriptions.sql
```

Benchmark 2: billing (run first, while the 200 subscriptions are due)

```
k6 run -e API_BASE=http://127.0.0.1:4100 --summary-export k6-billing-summary.json infra/load-test/k6/subscription-cron.js | tee k6-billing.txt
```

Benchmark 1, first run (script as committed; failed)

```
k6 run -e API_BASE=http://127.0.0.1:4100 infra/load-test/k6/checkout.js
```

Benchmark 1, second run (wrapper with `noCookiesReset: true`, see §4.1)

```
k6 run -e API_BASE=http://127.0.0.1:4100 --out csv=k6-checkout-raw.csv --summary-export k6-checkout-summary.json checkout-keep-cookies.js | tee k6-checkout.txt
```

Resource sampling (in a second terminal during the second run)

Every 2 seconds, record the time, CPU % and RSS (KB) of the API process, the PostgreSQL processes (summed) and the Redis process into `resource.csv`, with the columns `epoch, api_cpu, api_rss_kb, pg_cpu, pg_rss_kb, redis_cpu, redis_rss_kb`. An equivalent loop:

```
while true; do echo "$(date +%s),$(ps -o %cpu=,rss= -p "$(pgrep -f "node dist/main.js")" | awk '{print $1","$2}'),$(ps -axo %cpu=,rss=,command= | grep "[p]ostgres" | awk '{c+=$1;r+=$2} END {print c","r}'),$(ps -o %cpu=,rss= -p "$(pgrep -f "redis-server .*6380")" | awk '{print $1","$2}')" >> resource.csv; sleep 2; done
```

---

11. Appendix B: raw data files

File	Content
`k6-billing.txt`	k6 console output of Benchmark 2
`k6-billing-summary.json`	k6 summary export of Benchmark 2
`k6-checkout.txt`	k6 console output of Benchmark 1, second run
`k6-checkout-summary.json`	k6 summary export of Benchmark 1, second run
`k6-checkout-raw.csv`	Every k6 metric sample of Benchmark 1, second run (source of §4.4 per-endpoint rows)
`k6-status-breakdown.tsv`	Request count per endpoint and status (§4.3)
`k6-timeseries.tsv`	req/s, p50, p95 and checkout p95 per 10 s window (§4.5)

File	Content
`resource.csv`	87 resource samples (§6)
`resource-summary.txt`	Averages and maximums of the resource samples (§6)
`checkout-keep-cookies.js`	Wrapper used for the second checkout run (§4.1)